

# PREBORN: Construction Site Safety Monitoring System

Embedded Systems Workshop Project Report

Gyandeep Das

Harshil Soni

Samarth Jain

Swayam Goyal

December 10, 2025

## Abstract

Construction sites are among the most hazardous industrial environments, with a large proportion of injuries and fatalities caused by unsafe proximity between workers and heavy machinery. To address this challenge, we present PREBORN, a real-time safety monitoring system that performs on-device perception, depth estimation, and proximity assessment using embedded artificial intelligence. PREBORN deploys the YOLOv11 object detection network and the Depth-Anything-V2 depth estimation model on the Qualcomm AI Hub QIDK (Snapdragon 8 Gen 2) platform, optimized for heterogeneous acceleration using Hexagon DSP and NPU/HTP units. The system detects workers and multiple classes of construction equipment, computes three-dimensional spatial distances between them, and automatically identifies violations of safety thresholds. Results are streamed to a web dashboard, which provides interactive 2D/3D visualization and allows remote supervisors to monitor activity in real time. Safety incidents are logged through Firebase and communicated through immediate email alerts (planned to be implemented). Experimental evaluations demonstrate that the system achieves real-time inference performance while maintaining robust spatial accuracy. PREBORN provides a scalable and cost-effective approach to proactive site safety enforcement and illustrates the practical applicability of embedded edge AI in industrial monitoring environments.

## 1 Introduction

Construction sites remain one of the most hazardous work environments, with thousands of workers injured or killed annually due to unsafe interactions with heavy machinery. A significant portion of these incidents arises from workers entering dangerous proximity zones around moving vehicles, trucks, and equipment.

Conventional approaches rely on manual supervision, CCTV monitoring, or wearable tags. Manual supervision is error-prone and does not scale well, while CCTV monitoring lacks automated reasoning about distance and risk. Wearable technologies require strict compliance and additional hardware for each worker.

PREBORN addresses these limitations by providing automated, camera-based monitoring using embedded AI running directly on a Qualcomm AI Hub QIDK (Snapdragon 8 Gen 2) platform. The system

detects workers and machinery, estimates 3D distances from monocular depth, and flags safety violations in real time.

## 2 System Overview

The overall PREBORN system integrates perception, depth estimation, and embedded computation to provide automated real-time safety intelligence. The main functional stages are object detection, depth estimation, distance computation, safety classification, and dashboard visualization.

### 2.1 Application Workflow

The complete application workflow from image acquisition to safety alert generation is summarized below. This block-level description corresponds to the functional dataflow diagrams.

- 1) **Frame Acquisition:** The camera interfaced with the QIDK continuously provides video frames at a preconfigured resolution. Frames are buffered and passed into the processing pipeline.
- 2) **Pre-processing:** The incoming image is scaled and normalized. For YOLOv11, inputs are resized to the network-specific input dimensions, while for depth estimation, the tensor is preprocessed according to Depth-Anything-V2 requirements.
- 3) **Object Detection:** YOLOv11 performs multiclass detection (worker, truck, bulldozer, bike, car). Bounding boxes and confidence values are extracted.
- 4) **Depth Estimation:** The resized image is passed to Depth-Anything-V2, producing a pixel-level dense depth map. Normalized depth values are used for subsequent metric-space mapping.
- 5) **3D Distance Computation:** Using the detected bounding box centroids and calibrated intrinsic parameters, real-world distances between workers and machines are computed.
- 6) **Safety Classification:** If the computed distance falls below a configurable safety threshold, the frame is marked unsafe. Otherwise, it is labeled safe.
- 7) **Visualization and Overlay:** The detection boxes, depth overlays, and safe/unsafe labels are rendered as visual output for the Firebase-hosted dashboard.
- 8) **Alert Logging & Notifications:** Unsafe frames are stored in the Firebase database with timestamps and metadata. Email alerts are planned to be triggered automatically for supervisors.

### 2.2 Block Diagrams

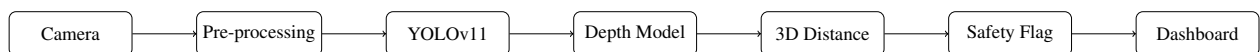


Figure 1: System-level block diagram for PREBORN.

Figure 2 shows the flow of data among major modules, ending in safety classification and dashboard updates.

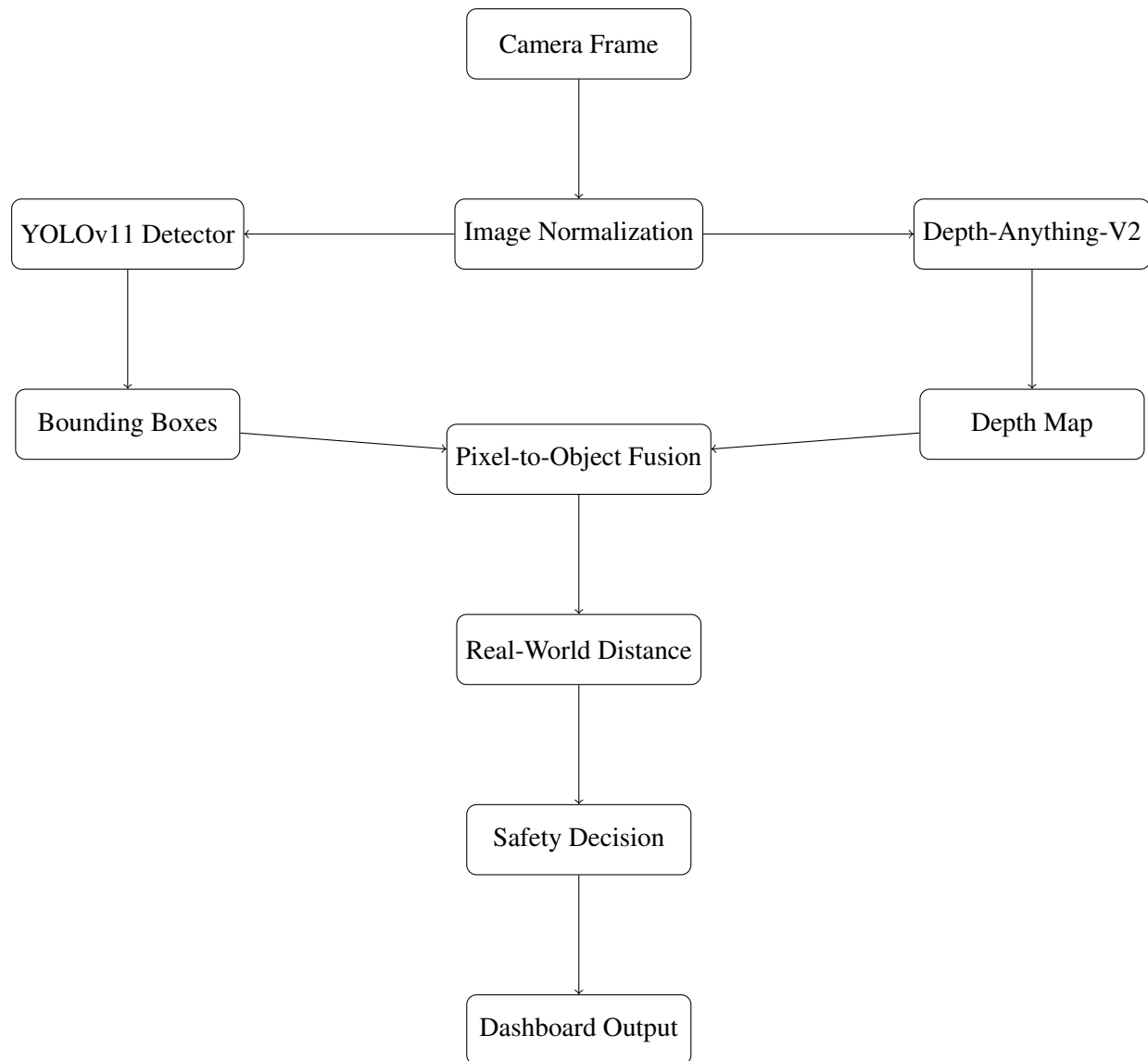


Figure 2: Detailed data-flow pipeline from camera input to dashboard output.

## 2.3 Application Workflow Visualizations

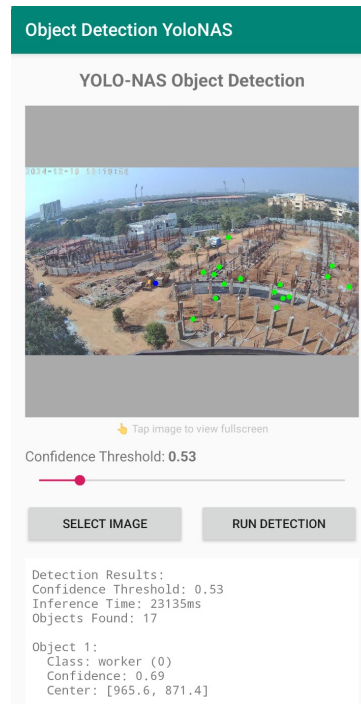


Figure 3: Application workflow stage: preprocessing and safety threshold setting.



Figure 4: Final app view using point-based visualization instead of full bounding boxes for mobile-friendly display.



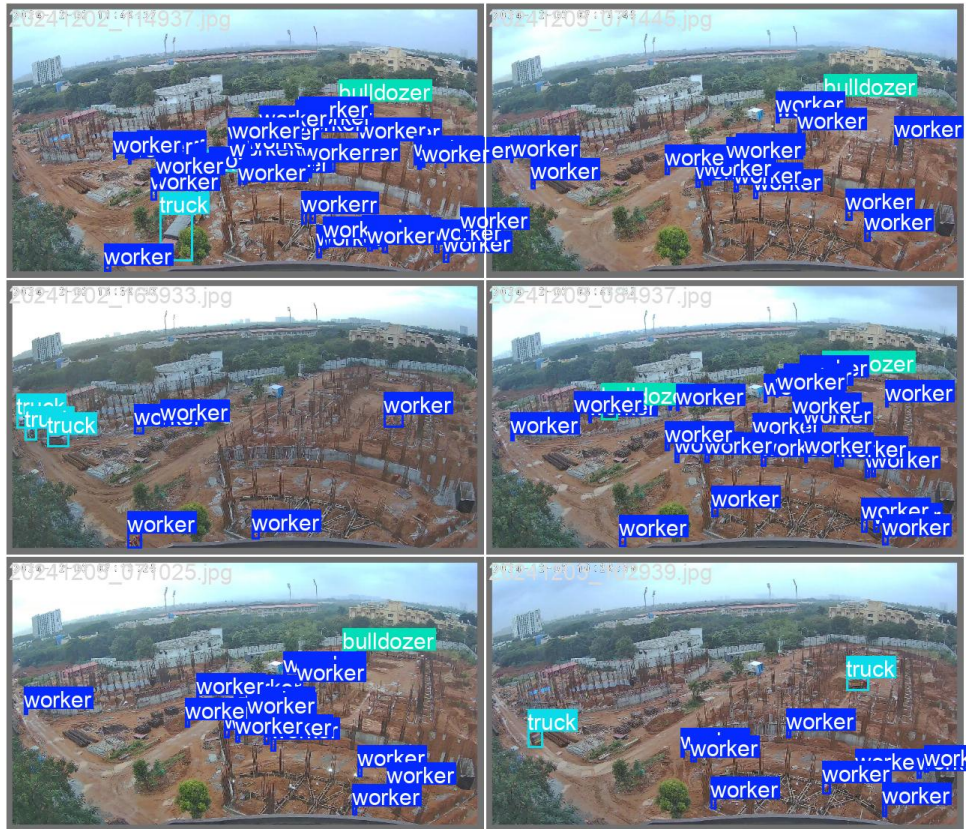


Figure 5: Workflow stage: YOLOv11 detection output with bounding boxes over workers and machinery.

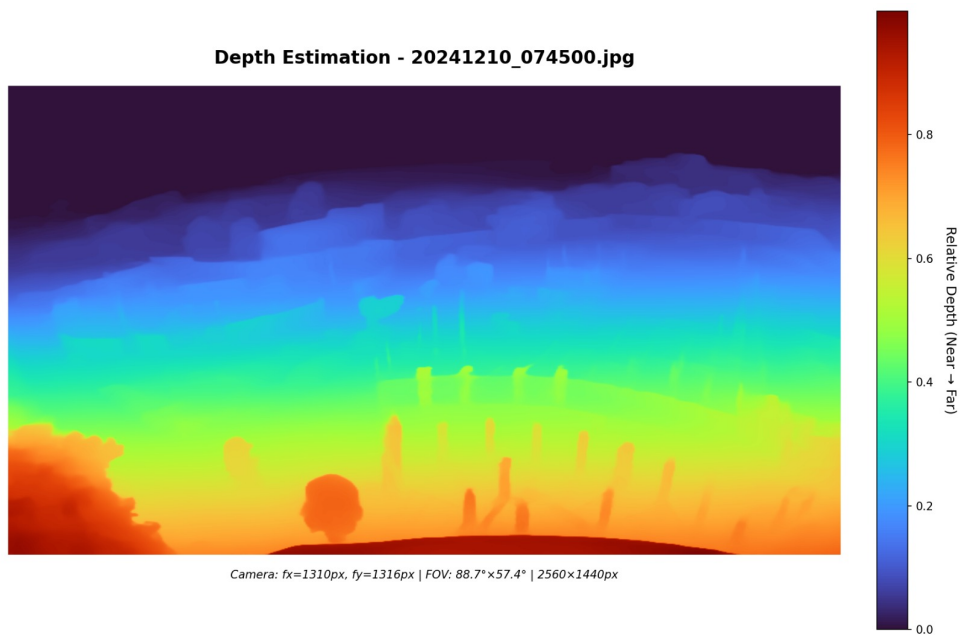


Figure 6: Workflow stage: depth estimation and real-world distance computation.

## 3 Methodology

### 3.1 Model Design Overview

Figure 7 shows the dual-branch model design wherein YOLOv11 provides semantic entity localization and Depth-Anything-V2 computes per-pixel depth.

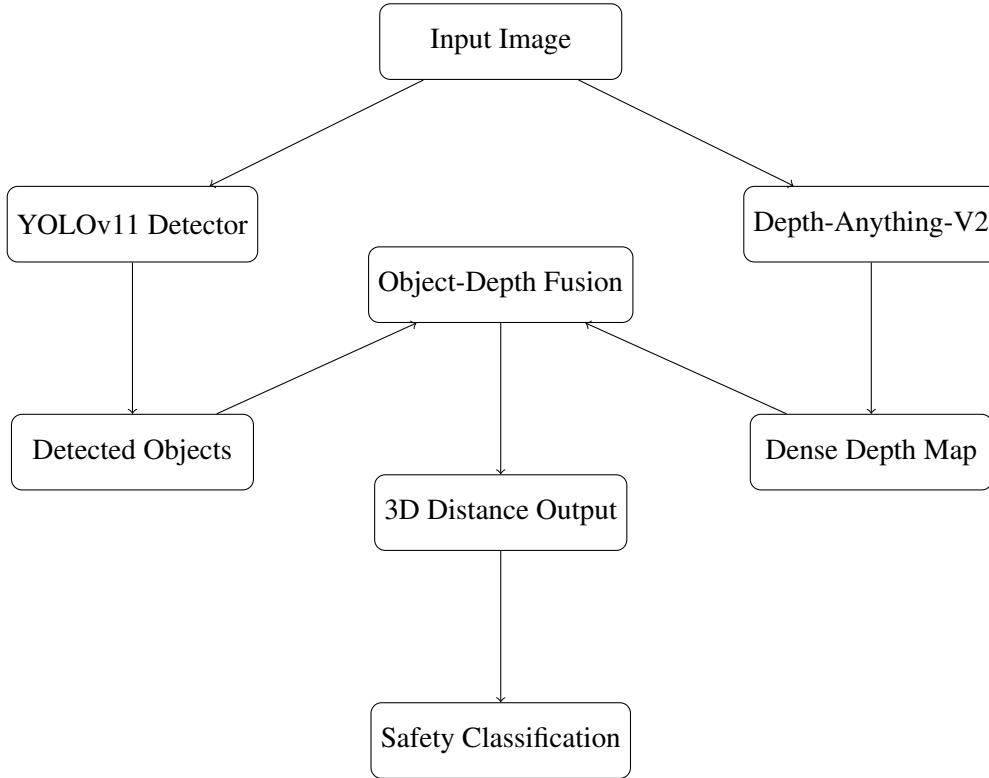


Figure 7: Dual-branch model architecture integrating detection and depth estimation.

### 3.2 Model Training Pipeline

The training pipeline consisted of multiple iterative steps aimed at improving detection robustness, reducing class ambiguity, and ensuring reliable operation in varied lighting and background conditions.

#### 3.2.1 Dataset Annotation and Refinement

A curated dataset was prepared by gathering construction-related images from open-source datasets and custom-captured frames. Annotation was performed using *LabelImg*, labeling the following classes:

- Worker
- Truck
- Bulldozer

- Bike
- Car

Manual re-annotation cycles were carried out to eliminate label noise and inconsistent bounding boxes.

### 3.2.2 Training Procedure

The YOLOv11 detector was trained using the annotated dataset. The training pipeline consisted of:

- Data augmentation (horizontal flips, brightness variation, random crops)
- Anchor refinement
- Focal-loss-based class balancing
- Multi-scale training

Training was performed in epochs, with periodic evaluation against the validation subset to monitor mean Average Precision (mAP) and avoid overfitting.

### 3.2.3 Hyperparameters

Typical training hyperparameters included:

- Learning rate:  $1 \times 10^{-3}$
- Batch size: 16
- Input resolution:  $640 \times 640$
- Optimizer: AdamW

Hyperparameters were adjusted iteratively based on loss stability and validation accuracy progression.

### 3.2.4 Performance Evaluation

Post-training, the following metrics were used to assess model quality:

- Precision and Recall
- mAP (Mean Average Precision)
- Confusion matrix behavior across classes

A key observation was that higher Recall was preferable to higher Precision for safety-critical applications, since missed detections (false negatives) were more dangerous than false alarms.

### 3.2.5 Model Export & Deployment

The best-performing model checkpoint was exported to ONNX. Static quantization and graph simplification were applied to facilitate deployment on the QIDK hardware accelerators. Benchmarking was repeated post-quantization to verify fidelity preservation.

### 3.3 Training and Calibration Visuals



Figure 8: Dataset annotation workflow using LabelImg for worker and machinery classes.

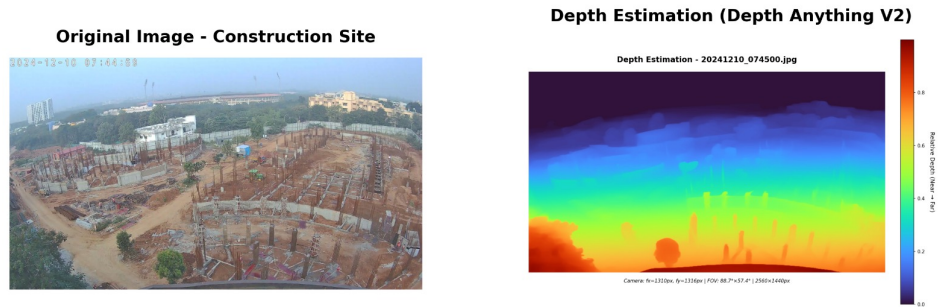


Figure 9: Depth-Anything-V2 qualitative depth estimation examples on construction scenes.



## Image + Depth Overlay with Principal Point



Figure 10: Camera calibration frames and estimated intrinsic parameters from checkerboard patterns.

## 4 Embedded Deployment on QIDK

### 4.1 Latency Comparison Across Compute Units

The inference time averaged across 100 frames is provided in Table 1.

Table 1: Latency performance for various hardware targets on the QIDK.

| Compute Unit | Latency (ms) |
|--------------|--------------|
| CPU          | 77.1         |
| GPU          | 45.9         |
| NPU          | 11.7         |

The NPU latency of 11.7 ms demonstrates that the optimized model comfortably meets real-time constraints, corresponding to a processing throughput above 85 FPS. This confirms that the QIDK acceleration stack efficiently supports the combined detection and depth workloads in an embedded deployment.

## 5 Experimental Evaluation

### 5.1 Object Detection Performance

Precision–Recall characteristics indicate superior recall, which is preferred for safety-critical monitoring. Figure 11 shows the PR curve, emphasizing the behavior of YOLOv11 in this domain.

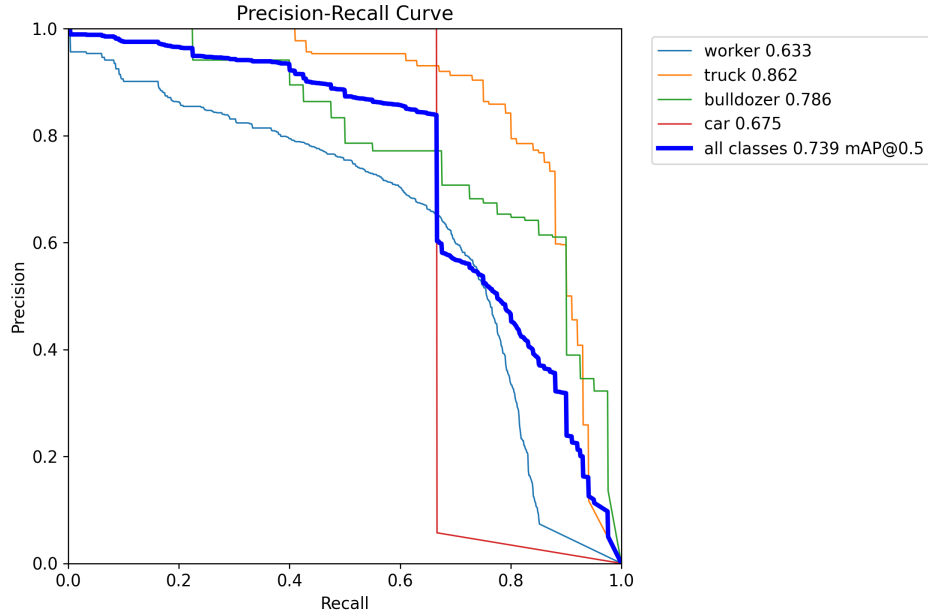


Figure 11: Precision–Recall curve obtained for the YOLOv11 detector on the validation set.

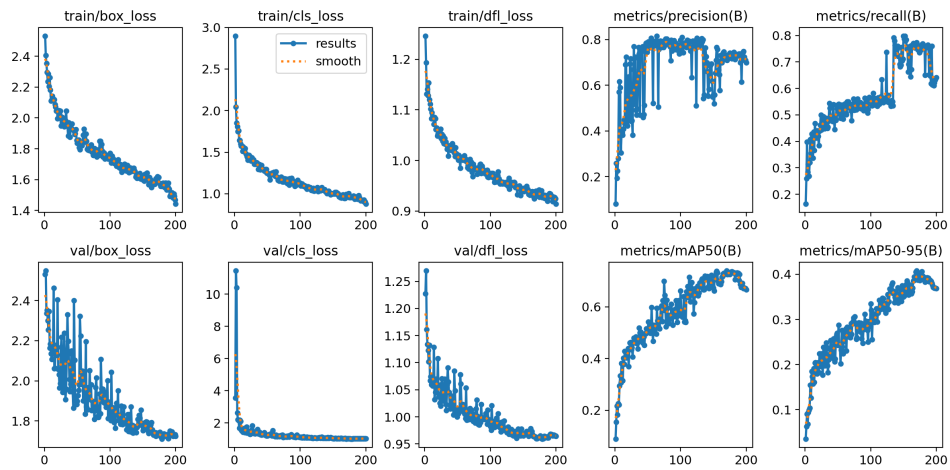


Figure 12: Training and validation curves: mAP and loss progression over epochs.

The confusion matrix for multiclass detection is provided in Table 2.

Table 2: Normalized confusion matrix for YOLOv11 detection.

|            | Worker | Truck | Bulldozer | Bike | Background |
|------------|--------|-------|-----------|------|------------|
| Worker     | 0.71   | 0.00  | 0.00      | 0.29 | 0.00       |
| Truck      | 0.00   | 0.84  | 0.00      | 0.00 | 0.16       |
| Bulldozer  | 0.00   | 0.02  | 0.82      | 0.00 | 0.16       |
| Bike       | 0.29   | 0.14  | 0.00      | 0.57 | 0.00       |
| Background | 0.00   | 0.02  | 0.01      | 0.00 | 0.97       |

### 5.1.1 Rationale for Choosing YOLOv11 over YOLOv8 and OWL-V2

Multiple candidate detection models were evaluated, including YOLOv8 and OWL-V2. The selection of YOLOv11 for deployment in PREBORN was based on empirical observations, hardware compatibility, and safety-driven performance requirements.

YOLOv11 exhibited the following advantages:

- **Higher Detection Reliability in Confusing Scenarios:** During testing, YOLOv8 demonstrated class-overlap issues between workers and machinery, particularly trucks. YOLOv11 provided improved localization stability and class discrimination, reducing false attribution errors.
- **Superior Recall—Critical for Safety Monitoring:** Recall was prioritized over precision since undetected hazards (false negatives) pose greater risks than false alarms. YOLOv11 consistently achieved higher recall scores than YOLOv8 in our evaluation.
- **Improved Backbone and Feature Aggregation:** YOLOv11 introduces architectural refinements and enhanced feature fusion leveraging spatial-channel interactions, improving robustness on cluttered construction scenes.
- **Better Deployment Characteristics on QIDK:** YOLOv11 ONNX quantization and layer fusion resulted in more favorable computational graph partitioning across CPU/GPU/NPU on QIDK, producing faster inference than YOLOv8 post-optimization.
- **Compatibility Limitations of OWL-V2:** OWL-V2 was investigated as an open-vocabulary alternative; however:
  - Model conversion to DLC was unsuccessful.
  - NNAPI offloading resulted in post-processing being executed solely on the CPU, drastically increasing latency.
  - Its lightweight transformer decoder led to higher computational variance and reduced consistency in spatial localization.
- **Model Size and Execution Efficiency:** YOLOv11 achieved an optimal balance between model size, computational load, and detection accuracy, making it feasible for real-time deployment on the QIDK edge platform.

## 5.2 Depth Model Calibration Results

Using 45 calibration frames, intrinsic parameters were computed. Table 3 summarizes calibrated intrinsic values.

Table 3: Calibrated intrinsic camera parameters.

| Parameter                      | Value            |
|--------------------------------|------------------|
| Focal Length ( $f_x, f_y$ )    | 1309.86, 1315.80 |
| Principal Point ( $c_x, c_y$ ) | 944.08, 542.44   |
| Resolution                     | 2560×1440        |
| Horizontal FOV                 | 88.7°            |
| Vertical FOV                   | 57.4°            |
| Calibration Frames             | 45               |

Figure 13 shows the depth distribution for typical scenes, where depth values concentrate within the normalized 0.2–0.5 band (excluding sky regions).

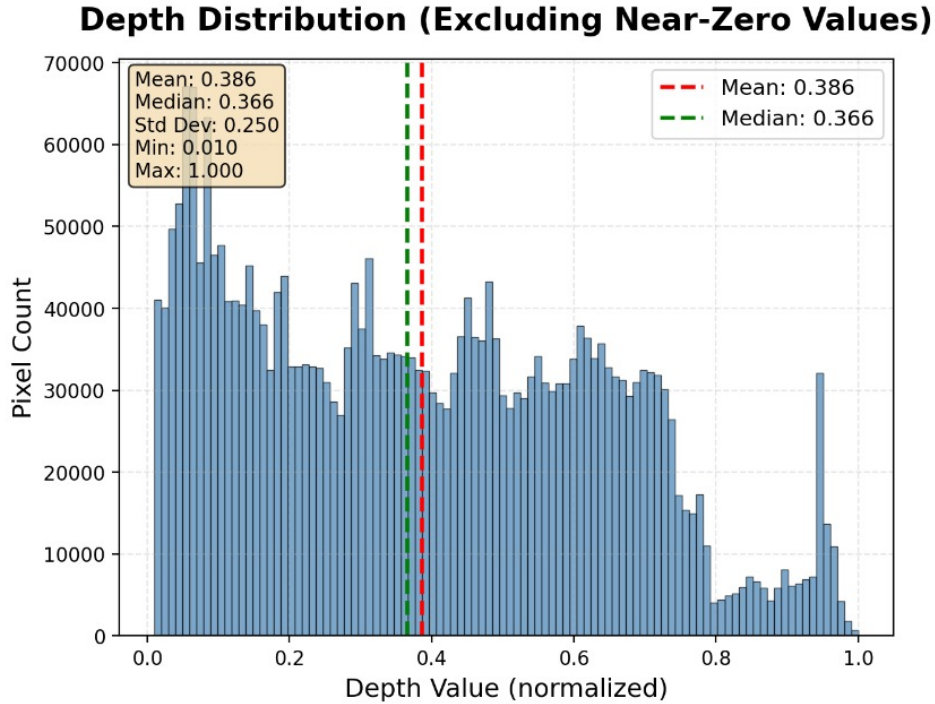


Figure 13: Depth distribution characteristics of Depth-Anything-V2 output over sampled frames.



### 5.3 Qualitative Results on Construction Scenes

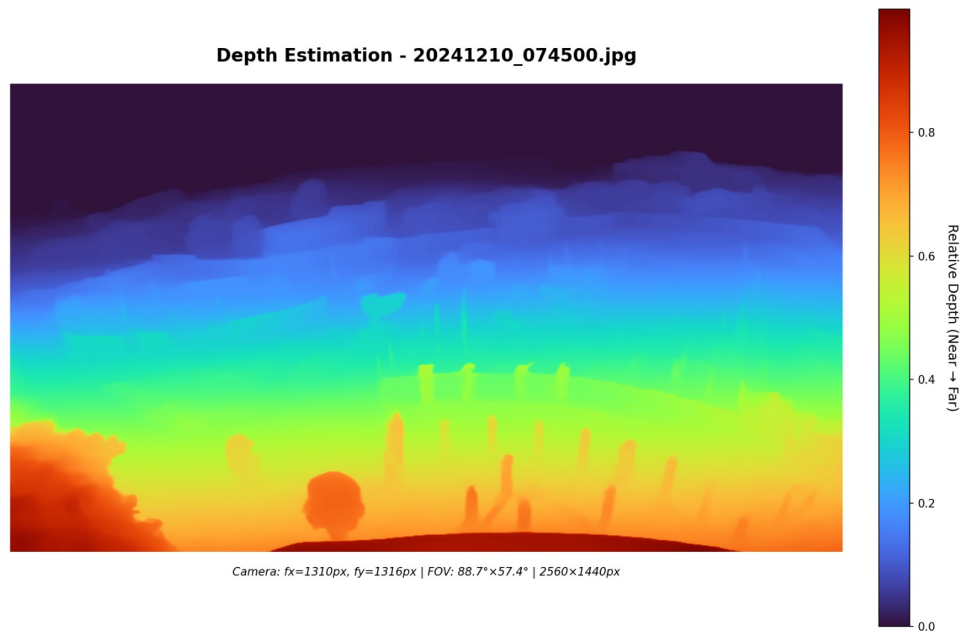


Figure 14: Depth map estimation on a typical construction frame using Depth-Anything-V2.

## 6 Dashboard and Cloud Integration

A real-time web-based dashboard was implemented using Firebase for hosting and backend services. The dashboard receives processed frames and associated metadata from the QIDK device and displays:

- Bounding boxes for workers and machinery.
- Numeric distance estimates and safety flags.
- Depth and 3D visualization panels.
- Lists of recent safety incidents with timestamps.

The interface supports dark and light themes and allows configurable safety thresholds.

## 6.1 Dashboard Visualizations

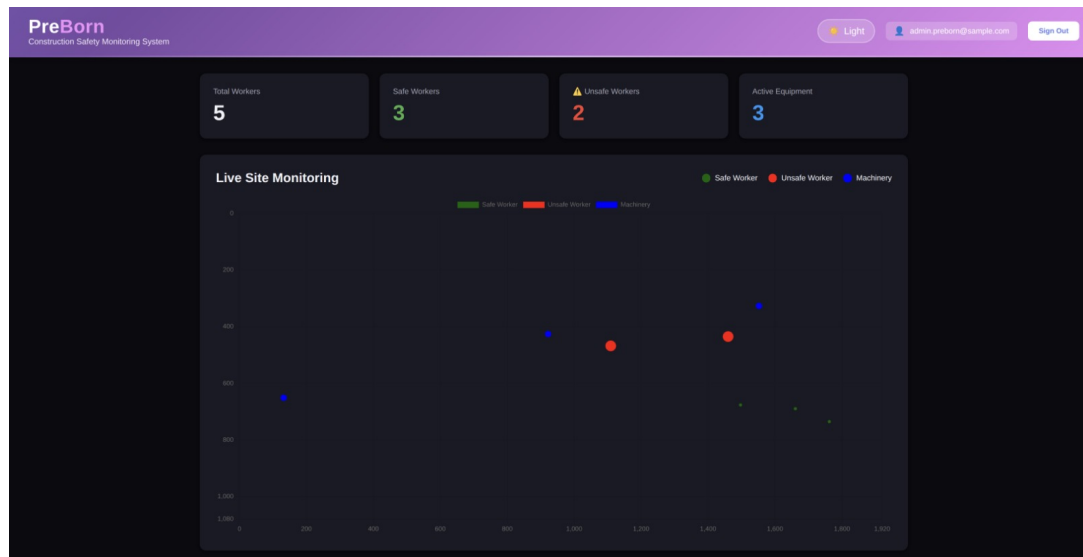


Figure 15: Firebase dashboard with real-time detection and distance visualization.

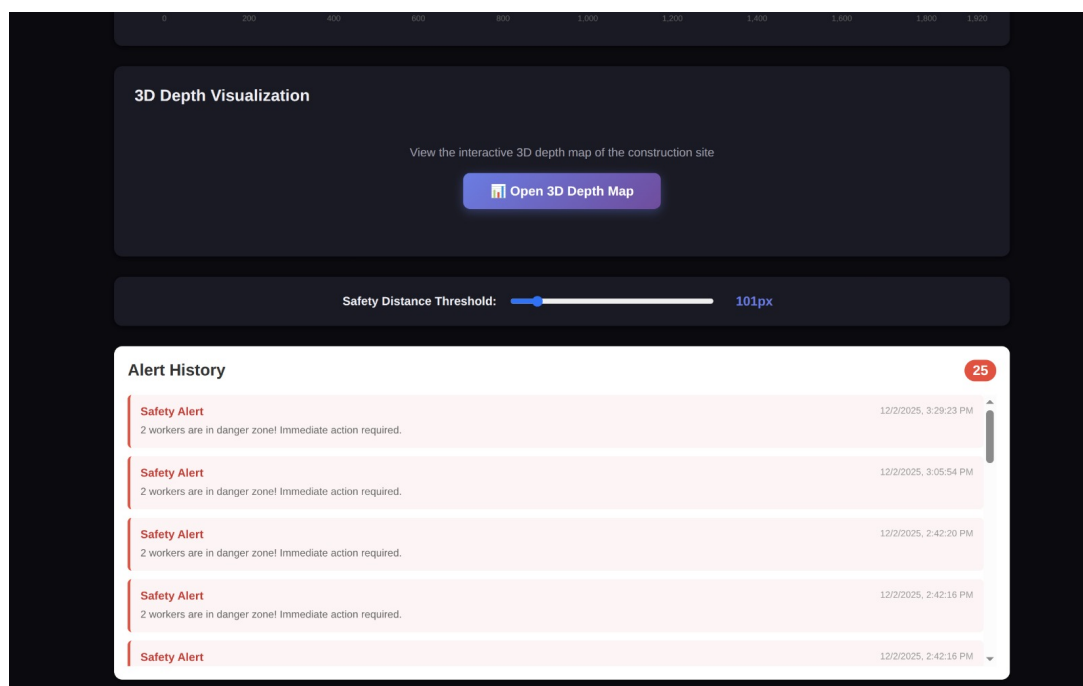


Figure 16: Alert system showing unsafe proximity notifications on the dashboard.

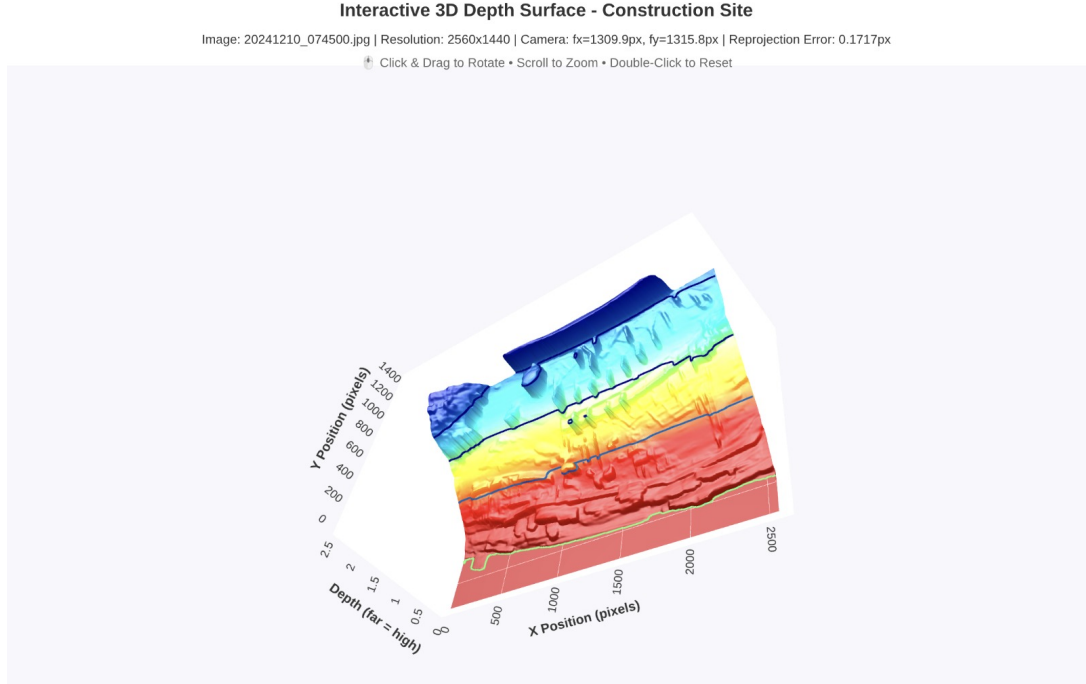


Figure 17: 3D surface plot of the depth environment generated from Depth-Anything-V2 outputs.

## 7 Conclusion and Future Work

PREBORN demonstrates that real-time worker-machine proximity monitoring is feasible on embedded hardware using modern computer vision and depth estimation models. By combining YOLOv11 detection with Depth-Anything-V2 depth maps and deploying the pipeline on the Qualcomm QIDK platform, the system achieves both high recall and low latency, which are essential for safety-critical applications.

Future work includes:

- Expanding the dataset using public construction datasets and synthetic augmentation.
- Integrating personal protective equipment (PPE) detection such as helmets and vests.
- Exploring multi-camera setups for improved 3D localization and occlusion handling.
- Implementing and validating the automated email alerting pipeline with supervisors in real deployments.